

フロントエンド側のドメイン書き換え

GitHub Codespacesからローカル環境に移し替えたことで、これまでは一時的な公開プロキシドメインを暫定的にハードコーディングしていた箇所に、動的なドメインを動作環境に応じて当てはめることが必要となりました。

通常、Vueの接続先ドメインは、バックエンドから送ってもらうのではなく、フロントエンド（Vite）側の環境変数を使って切り替えるのが一般的です。

（バックエンド接続用のドメインを接続前のバックエンドから取得することは原理的に難しい）

今回は、Vue/Vite側で環境変数を導入してURLを動的に切り替える方法と、それに伴って必要となるバックエンド（Spring Boot）側の修正について解説します。

解決のための全体像

1. フロントエンド（Vue/Vite）：環境変数ファイル（`.env`）を作り、環境ごとのURLを自動で切り替えるようにする。
2. バックエンド（Spring Boot）：ローカル環境（`localhost:5173` など）からの接続を許可するため、CORS（Cross-Origin Resource Sharing）の設定を `application.properties` とJavaで修正する。

フロントエンド（Vite）で環境変数を導入する

Viteには、開発環境と本番環境（あるいはCodespaces環境）で記述を自動で切り替える便利な仕組みが備わっています。

1. `.env` ファイルの作成

`frontend` フォルダの直下（`package.json` と同じ階層）に、以下の2つのファイルを新規作成します。

```
# `\.env.development`（ローカル開発用の設定ファイル）  
VITE_API_BASE_URL=http://localhost:8080
```

```
# `\.env.production` (Codespacesや本番用の設定ファイル)  
VITE_API_BASE_URL=https://ubiquitous-spork-4vq65g5rr79c5j6q-8080.app.github.dev
```

(※Viteのルールで、頭に `VITE_` をつけた変数だけがVueファイル側から読み込めます)

2. Vueファイルの記述を変更する

提示いただいた `fetch` の部分を、この環境変数を使うように書き換えます。

```
try {  
  // 固定URLを環境変数 (import.meta.env.VITE_API_BASE_URL) に置き換える  
  const response = await fetch(`${import.meta.env.VITE_API_BASE_URL}/api/users/register`, {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({  
      userName: userName.value,  
      userId: userId.value,  
      password: password.value  
    })  
  });  
}
```

これで、ローカルで `npm run dev` をしている時は自動的に `http://localhost:8080` が適用され、コードを書き換える必要がなくなります。

バックエンド（Spring Boot）でCORS接続を許可する

ドメインが `localhost` 同士に変わったことで、ブラウザのセキュリティ機能（CORS）に引っかかる可能性が高くなります。バックエンド側で「ローカルのフロントエンドからのアクセスを許可する」設定を入れましょう。

1. `application.properties` にフロントエンドのURLを記述

`backend/src/main/resources/application.properties` に以下を追記します。

```
# フロントエンド（Vite）のローカルサーバーのURLを指定
cors.allowed-origins=http://localhost:5173
```

(※Codespacesに戻る可能性がある場合は、ここを `,` 区切りでCodespacesのフロントエンドURLを追加すればOKです)

補足: カスタムプロパティ `cors.allowed-origins` について

今回の `cors.allowed-origins` のような独自の変数（カスタムプロパティ）を `application.properties` 内で記述すると、VS Codeなどのエディタは「そんな名前の設定は知らないよ」という意味で警告（波線や、不明な変数という表示）を出してしまいます。

とはいえ、**警告が出ていてもそのままJava側で `@Value("${cors.allowed-origins}")` を使って呼び出すことは可能**です。Spring Bootは、自分が知らない独自の変数であっても、Javaコード側から明示的に指定されれば「ただの文字列の変数」としてちゃんと読み込んでくれます。

もし、この「不明な変数」という警告をきれいに消して、Spring Bootの正式な設定としてスマートに組み込みたい（有効化したい）場合は、Java側で**カスタム設定クラス**を作って紐付けるのが最も美しく、実務でもよく使われるベストプラクティスです。

その具体的な設定方法を解説します。

独自変数を「有効な設定」として定義する方法

Java側に `@ConfigurationProperties` というアノテーションをつけたクラスを1つ用意することで、`cors.allowed-origins` を「システムが公認した正しい変数」として有効化できます。

バックエンドのソースコード（例えば `config` などのパッケージ内）に、新しく `CorsProperties.java` というファイルを作成します。

```
package com.example.demo.config; // ご自身のパッケージ名に合わせてください

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.context.annotation.Configuration;

@Configuration
// application.properties の「cors」で始まる設定をこのクラスにマッピングする
@ConfigurationProperties(prefix = "cors")
public class CorsProperties {

    // cors.allowed-origins の値がここに自動で注入されます
    private String allowedOrigins;
```

```
// ゲッターとセッターが必要
public String getAllowedOrigins() {
    return allowedOrigins;
}

public void setAllowedOrigins(String allowedOrigins) {
    this.allowedOrigins = allowedOrigins;
}
}
```

設定クラスを作ったら、コントローラー側（UserController など）では @Value を使うのではなく、作成した CorsProperties クラスをインジェクション（注入）して利用します。

```
@RestController
@RequestMapping("/api/users")
@CrossOrigin(origins = "#{corsProperties.allowedOrigins}") // 設定クラスから動的に取得
public class UserController {
```

```
// または、以下のようにDIして細かく制御することも可能です
private final CorsProperties corsProperties;

public UserController(CorsProperties corsProperties) {
    this.corsProperties = corsProperties;
}

// 省略...
}
```

一番手っ取り早く警告を無視・解決したい場合

「設定クラスを新しく作るのはちょっとな...」と感じる場合は、以下の方法でも全く問題ありません。

警告を無視してそのまま使う

前述の通り、エディタが「不明な変数」と警告を出していても、Spring Bootのアプリケーション自体は正常に起動し、Javaの `@Value("${cors.allowed-origins}")` で値を読み込むことができます。開発中の動作検証だけであれば、警告はそのままにして進めてしまっても実害はありません。

既存の「spring.」の枠組みに乗っかる

もし独自変数を作るのをやめたい場合は、Spring Bootが最初から内部で持っているCORSの仕組みを利用する方法もあります。

しかしこの場合は結局Java側で細かく制御することになるため、今回のように `cors.allowed-origins` という名前のまま、上記の「設定クラス（`CorsProperties`）」を作っただけなのが、コードの可読性の観点からも一番スッキリしてお勧めです。

エディタの警告が気層な場合は、ぜひ `CorsProperties` クラスを作成して、システムに「公認」させてみてください。

2. Javaのコントローラー（または設定クラス）で読み込む

ユーザー登録API（`/api/users/register`）を受け持っているバックエンドのコントローラー（`@RestController`）のクラス、またはメソッドに対して、以下のように `@CrossOrigin` アノテーションを付与してプロパティを読み込ませます。

```
@RestController
```

```
@RequestMapping("/api/users")
// application.properties の値を取り出してCORS許可originに設定
@CrossOrigin(origins = "${cors.allowed-origins}")
public class UserController {

    @PostMapping("/register")
    public ResponseEntity<?> registerUser(@RequestBody UserDto userDto) {
        // 登録処理...
    }
}
```

もしプロジェクト全体で一括してCORSを設定している構成であれば、`WebMvcConfigurer` を実装した設定クラスの中で、`@Value("${cors.allowed-origins}")` を使って設定を反映させる形になります。